

Strategies for Proving Greedy Algorithms are Correct

CMSC 27200: Theory of Algorithms

Two important proof strategies for proving that a greedy algorithm is correct are as follows:

Greedy stays ahead: Show that at each step, the greedy solution is either ahead of or tied with any other potential solution.

Exchange argument: Show that for any solution which is not the greedy solution, we can transform this solution into the greedy solution by iteratively exchanging pairs of elements and these exchanges can only improve the solution's performance.

Remark 0.1. *The proof that Dijkstra's algorithm for the shortest path problem is correct and the proofs that Kruskal's algorithm and Prim's algorithm for the minimum spanning tree problem are correct actually use a different proof strategy, which is to show that these algorithms are correct at each step. That said, the homework and exam problems on greedy algorithms will use the two strategies above.*

We now describe how to use both of these strategies in more detail.

1 Proving that a Greedy Algorithm Stays Ahead

To prove that a greedy algorithm is optimal by showing that it stays ahead of or tied with all other possible solutions, we do the following:

1. Give a **measure of progress** by which the greedy algorithm stays ahead (or tied).
2. Prove that according to this measure, the greedy algorithm stays ahead of or tied with all other possible solutions. The key step for this is to use the description of the greedy algorithm to show that **if the greedy algorithm is ahead or tied at the beginning of the current step then it will be ahead or tied at the end of the current step.**

Remark 1.1. *It may be convenient to treat the first step of the algorithm separately, though it is often not necessary to do so.*

We illustrate this proof strategy with two examples.

1.1 Example 1: Kleinberg-Tardos Chapter 4 Solved Exercise 1

Restatement of the problem: Let's say that we are traveling on a line with rest stops at positions $x_1, \dots, x_n$ and we want to reach the final stop x_n in as few days as possible. However, the maximum distance we can travel each day is d . Which rest stops should we stop at in order to achieve this goal?

In other words, we want to find a schedule $S = (i_1, \dots, i_k)$ such that $k = |S|$ is minimized and:

1. $x_{i_1} \leq d$ (we travel at most distance d on the first day).
2. For all $j \in [k - 1]$, $x_{i_{j+1}} - x_{i_j} \leq d$ (we travel at most distance d on all subsequent days).

Note: We assume that $0 < x_1 < x_2 < \dots < x_n$, $x_1 \leq d$, and $x_{i+1} \leq x_i + d$ for all $i \in [n - 1]$.

A greedy algorithm for this problem is to **go as far as we can each day**. More precisely, this algorithm works as follows:

1. Initialization: We take $i_1 = \max \{i : i \in [n], x_i \leq d\}$.
2. Iterative step: If $(i_1, \dots, i_j)$ is our schedule for the first j days and $i_j < n$ then we take $i_{j+1} = \max \{i : i \in [n], x_i - x_{i_j} \leq d\}$. If $i_j = n$ then we stop and output $S = (i_1, \dots, i_j)$.

We can prove that this algorithm is correct as follows:

1. Measure of progress: Given a schedule $S = (i_1, \dots, i_k)$, observe that i_j is **the number of rest stops reached after j days** and this is a good measure of progress.
2. Proving that the greedy algorithm stays ahead of or tied with all other possible solutions: Let $S_{\text{greedy}} = (i_1, \dots, i_k)$ be the solution given by this greedy algorithm and let $S' = (i'_1, \dots, i'_{k'})$ be another solution. We need to show that $k \leq k'$. To show this, it is sufficient to show that for all $j \in [k]$, i'_j exists and $i_j \geq i'_j$.

We can prove that $i_j \geq i'_j$ for all $j \in [k]$ by induction on j . For the base case, observe that $i_1 = \max \{i : i \in [n], x_i \leq d\}$ so since $x_{i'_1} \leq d$, we must have that $i_1 \geq i'_1$, as needed.

For the inductive step, assume that $i_j \geq i'_j$ for some $j < k$. Since $j < k$, $i'_j \leq i_j < n$ so i'_{j+1} must exist. Since $x_{i'_{j+1}} - x_{i'_j} \leq x_{i_{j+1}} - x_{i_j} \leq d$ and $i_{j+1} = \max \{i : i \in [n], x_i - x_{i_j} \leq d\}$, we must have that $i_{j+1} \geq i'_{j+1}$, as needed.

Remark 1.2. The argument for the inductive part can be rephrased verbally as follows. Since $i_j \geq i'_j$, **all of the rest stops available to S' for day $j + 1$ are also available to S_{greedy}** (unless S_{greedy} has already passed these rest stops). Since the greedy algorithm **goes as far as possible each day**, $i_{j+1} \geq i'_{j+1}$.

Remark 1.3. The distance x_{i_j} traveled after j days would also be a good measure of progress to use. However, looking at which day a given rest stop is reached on would not be a good measure of progress to use because it is missing the key information of where the schedule started on that day.

1.2 Example: Interval Scheduling

Recall the interval scheduling problem: Given n intervals $\{[a_i, b_i] : i \in [n]\}$, find a schedule $S = (i_1, \dots, i_k)$ such that for all $j \in [|S| - 1]$, $b_{i_j} \leq a_{i_{j+1}}$ (no job starts before the previous job finishes) and $k = |S|$ is maximized.

The greedy algorithm for this problem is to iteratively go through the jobs which are still available and **take the job which finishes first**. More precisely, this algorithm works as follows.

1. Initialization: Take i_1 so that $b_{i_1} = \min \{b_i : i \in [n]\}$ (i.e., the first job finishes as soon as possible). If there is more than one such i_1 then we can choose any of them.
2. Iterative step: If $(i_1, \dots, i_j)$ is our schedule for the first j days and there is at least one i such that $a_i \geq b_{i_j}$ then we take i_{j+1} so that $a_{i_{j+1}} \geq b_{i_j}$ and $b_{i_{j+1}} = \min \{b_i : i \in [n], a_i \geq b_{i_j}\}$ (i.e., the $(j + 1)$ th job finishes as soon as possible while not starting before the j th job finishes). If there is more than one such i_{j+1} then we can choose any of them.

We can prove that this algorithm is correct as follows:

1. Measure of progress: Given a schedule $S = (i_1, \dots, i_{|S|})$, observe that b_{i_j} is **the time when S finishes its j th job** and this is a good measure of progress.
2. Proving that the greedy algorithm stays ahead of or tied with all other possible solutions: Let $S_{\text{greedy}} = (i_1, \dots, i_k)$ be the greedy solution and let $S' = (i'_1, \dots, i'_{k'})$ be another solution. We need to show that for all $j \in [k]$, either $i'_{k'}$ does not exist or $b_{i_j} \leq b_{i'_j}$.

In the greedy algorithms lecture notes, we proved this using induction. For variety, we instead prove this using a proof by contradiction.

Assume that there is a $j \in [k]$ such that i'_j exists and $b_{i'_j} < b_{i_j}$ and consider **the first such j** . We cannot have that $j = 1$ as $b_{i_1} = \min \{b_i : i \in [n]\}$ so $b_{i'_1} \geq b_{i_1}$. Thus, we can assume that $j > 1$.

Since j is **the first** index in $[k]$ such that $b_{i'_j} < b_{i_j}$, we must have that $b_{i'_{j-1}} \geq b_{i_{j-1}}$. Since $a_{i'_j} \geq b_{i'_{j-1}} \geq b_{i_{j-1}}$ and $b_{i_j} = \min \{b_i : i \in [n], a_i \geq b_{i_{j-1}}\}$, $b_{i_j} \leq b_{i'_j}$ which contradicts our assumption that $b_{i'_j} < b_{i_j}$.

Remark 1.4. The argument for $j > 1$ can be rephrased verbally as follows. Since j is **the first** index in $[k]$ where $b_{i'_j} < b_{i_j}$, $b_{i'_{j-1}} \geq b_{i_{j-1}}$. This implies that **all of the jobs which are available to S' for the $(j + 1)$ th job are also available to S_{greedy}** . Since S_{greedy} **takes the job which finishes soonest**, we must have that $b_{i_j} \leq b_{i'_j}$ which contradicts our assumption that $b_{i'_j} < b_{i_j}$.

Remark 1.5. While the number of jobs completed by a given time is a plausible measure of progress, it does not work well for the analysis because it is missing the key information of when the current job will finish.

Remark 1.6. For these examples, we focused on proving that the algorithms are correct rather than how to implement them efficiently. In particular, we used max and min operations when describing the algorithms. In order to describe how to implement these algorithms efficiently, we need to describe how to implement these algorithms so that we only go through each rest stop/interval once.

1.3 Summary: Key Elements for Proving a Greedy Algorithm Stays Ahead

We now summarize the key elements for proving that a greedy algorithm stays ahead. As described informally at the beginning of the section and illustrated by the two examples above, we show that a greedy algorithm stays ahead by doing the following:

1. Provide a measure $f_j(S)$ for the progress made by a solution S after step j .
2. Show directly that for any other solution S' , $f_1(S_{\text{greedy}}) \geq f_1(S')$ (i.e., the greedy algorithm is ahead or tied after the first step).
3. Show that for all other solution S' and all $j \geq 2$, if $f_{j-1}(S_{\text{greedy}}) \geq f_{j-1}(S')$ then $f_j(S_{\text{greedy}}) \geq f_j(S')$ (i.e., if the greedy algorithm was ahead or tied after the $(j - 1)$ th step then it stays ahead or tied after the j th step).

Remark 1.7. *It is often possible to combine the second and third steps into one step by starting from $j = 0$ and setting $f_0(S) = 0$ for all S .*

With these ingredients, it is not hard to deduce that the greedy algorithm stays ahead. More precisely, the following statement can be proved either by induction or by a proof by contradiction.

Lemma 1.8. *Let S_{greedy} be the solution given by the greedy algorithm. If the above statements hold then for all other solutions S' and all $j \geq 1$, $f_j(S_{\text{greedy}}) \geq f_j(S')$.*

Proof by induction. Let S' be another solution. For the base case, observe that by the second statement, $f_1(S_{\text{greedy}}) \geq f_1(S')$. For the inductive step, assume that $f_{j-1}(S_{\text{greedy}}) \geq f_{j-1}(S')$. By the third statement, if $f_{j-1}(S_{\text{greedy}}) \geq f_{j-1}(S')$ then $f_j(S_{\text{greedy}}) \geq f_j(S')$ so we have that $f_j(S_{\text{greedy}}) \geq f_j(S')$, as needed. $\square$

Proof by contradiction. Let S' be another solution. Assume there is an index $j \in \mathbb{N}$ such that $f_j(S_{\text{greedy}}) < f_j(S')$ and choose j to be the first such index. By the second statement, $f_1(S_{\text{greedy}}) \geq f_1(S')$ so we cannot have that $j = 1$. Thus, we can assume that $j > 1$.

Since $j \in \mathbb{N}$ is the first index such that $f_j(S_{\text{greedy}}) < f_j(S')$, $f_{j-1}(S_{\text{greedy}}) \geq f_{j-1}(S')$. By the third statement, if $f_{j-1}(S_{\text{greedy}}) \geq f_{j-1}(S')$ then $f_j(S_{\text{greedy}}) \geq f_j(S')$ so we have that $f_j(S_{\text{greedy}}) \geq f_j(S')$. This contradicts our assumption that $f_j(S_{\text{greedy}}) < f_j(S')$. $\square$

2 Exchange Arguments

To prove that a greedy algorithm is optimal using an exchange argument, we do the following:

1. Given a solution S' which is not the greedy solution, find an **exchange of two elements of S'** which makes S' **closer to the greedy solution**.
2. Show that this exchange **can only improve S'** .
3. Conclude that the greedy solution is optimal. The intuition for this is that if there is another optimal solution S' , we can perform a sequence of exchanges to S' to get closer and closer to the greedy solution until we reach the greedy solution. Since these exchanges can only improve our solution, the greedy solution must also be optimal. While this argument isn't quite rigorous, as we describe below, it is not hard to make this argument rigorous.

We illustrate this proof strategy with two examples. Before giving these examples, we describe the lexicographic/alphabetical order (which is what is used in the dictionary) and inversions as these concepts are useful for verifying that exchanges bring us closer to the greedy solution.

2.1 The Lexicographic Order and Inversions

Definition 2.1. Given two sequences $S = (i_1, \dots, i_n)$ and $S' = (i'_1, \dots, i'_{n'})$, we say that S comes before S' in the lexicographic/alphabetical order if one of the following holds:

1. There is a $k \in [n]$ such that $i_k < i'_k$ and for all $j \in [k - 1]$, $i_j = i'_j$.
2. $n' > n$ and $i'_j = i_j$ for all $j \in [n]$.

Example 2.2. Given the sequences $(4, 1, 6, 2, 7, 8)$, $(1, 9, 8, 7, 6, 5)$, $(4, 1, 5, 9, 7, 3)$, $(2, 6, 4, 1, 5, 9)$, $(2, 5, 8, 1, 4, 3)$, and $(4, 1, 6)$, the lexicographic/alphabetical order for these sequences is $(1, 9, 8, 7, 6, 5)$, $(2, 5, 8, 1, 4, 3)$, $(2, 6, 4, 1, 5, 9)$, $(4, 1, 5, 9, 7, 3)$, $(4, 1, 6)$, and $(4, 1, 6, 2, 7, 8)$.

Definition 2.3. Given a sequence $S = (i_1, \dots, i_n)$, an inversion is a pair of indices j, j' such that $j < j'$ but $i_j > i_{j'}$.

Example 2.4. The sequence $S = (5, 2, 4, 1, 3)$ has 7 inversions as 5 comes before 2, 4, 1, and 3, 2 comes before 1, and 4 comes before 1 and 3.

Proposition 2.5. Given a sequence $S = (i_1, \dots, i_n)$ and an index $j \in [n - 1]$ such that $i_{j+1} < i_j$, the sequence $S' = (i_1, \dots, i_{j-1}, i_{j+1}, i_j, i_{j+2}, \dots, i_n)$ obtained by taking S and swapping i_j and i_{j+1} has one fewer inversion than S .

2.2 Example 1: Minimizing Maximum Lateness

Recall the minimizing maximum lateness problem. We are given n jobs, each of which has a length $l_i > 0$ and a deadline d_i (and can be scheduled at any time $t \geq 0$). We are then asked to complete the jobs one by one so that the maximum lateness of any single job is minimized.

As described in the lecture notes, we can state this problem mathematically using the following definition.

Definition 2.6. Given a schedule $S = (i_1, \dots, i_n)$ where $(i_1, \dots, i_n)$ is a permutation of $[n]$, define $t_S(i_k) = \sum_{j=1}^k l_{i_j}$ to be the time at which job i_k is finished.

Problem 2.7. Find a schedule $S = (i_1, \dots, i_n)$ where $(i_1, \dots, i_n)$ is a permutation of $[n]$ such that $\max_{j \in [n]} \{t_S(i_j) - d_{i_j}\}$ is minimized.

The greedy algorithm for this problem is to **complete the jobs in order of their deadlines**.

We can prove that this greedy algorithm is optimal as follows. For simplicity, assume that the jobs are labeled so that $d_1 \leq \dots \leq d_n$ and the greedy solution is $S_{\text{greedy}} = (1, 2, \dots, n)$.

1. Finding an exchange: Given a solution $S' = (i'_1, \dots, i'_n)$ which is not the greedy solution, **let j be the first index such that $i'_{j+1} < i'_j$ and swap i'_j and i'_{j+1}** . This gives the solution $S'' = (i'_1, \dots, i'_{j-1}, i'_{j+1}, i'_j, i'_{j+2}, \dots, i'_n)$.

2. Showing that the exchange can only improve S' : We make the following observations:

- (a) For all $j' \in [n] \setminus \{j, j+1\}$, $t_{S''}(i'_{j'}) = t_{S'}(i'_{j'})$.
- (b) $t_{S''}(i'_{j+1}) < t_{S''}(i'_j) = t_{S'}(i'_{j+1})$.
- (c) Since $d_{i'_{j+1}} \leq d_{i'_j}$, $t_{S''}(i'_j) - d_{i'_j} \leq t_{S'}(i'_{j+1}) - d_{i'_{j+1}}$.
- (d) Since $t_{S''}(i'_{j+1}) < t_{S'}(i'_{j+1})$, $t_{S''}(i'_{j+1}) - d_{i'_{j+1}} \leq t_{S'}(i'_{j+1}) - d_{i'_{j+1}}$.
- (e) Putting the third and fourth observations together,

$$\max \{t_{S''}(i'_j) - d_{i'_j}, t_{S''}(i'_{j+1}) - d_{i'_{j+1}}\} \leq t_{S'}(i'_{j+1}) - d_{i'_{j+1}} \leq \max \{t_{S'}(i'_j) - d_{i'_j}, t_{S'}(i'_{j+1}) - d_{i'_{j+1}}\}.$$

Combining this with the first observation, $\max_{j \in [n]} \{t_{S''}(i'_j) - d_{i'_j}\} \leq \max_{j \in [n]} \{t_{S'}(i'_j) - d_{i'_j}\}$, as needed.

3. Concluding that S_{greedy} is optimal: It would be acceptable to say something like “Since any other optimal solution can be made closer to the greedy solution without affecting its optimality, the greedy solution must be optimal.” However, this is not quite rigorous.

One way to rigorously prove that the greedy solution is optimal is to use a proof by contradiction using the lexicographic order.

Proof by contradiction using the lexicographic order. Assume S_{greedy} is not an optimal solution and let $S' = (i'_1, \dots, i'_n)$ be the optimal solution which comes first in the lexicographic order. Letting j be the first index such that $i'_{j+1} < i'_j$, applying the exchange described above gives a solution $S'' = (i'_1, \dots, i'_{j-1}, i'_{j+1}, i'_j, i'_{j+2}, \dots, i'_n)$. Observe that S'' is optimal and comes before S' in the lexicographic order, which contradicts the assumption that S' is the optimal solution which comes first in the lexicographic order. $\square$

Another way to prove that the greedy algorithm is optimal is to use a proof by contradiction using the number of inversions.

Proof by contradiction using the number of inversions. Assume S_{greedy} is not an optimal solution and let $S' = (i'_1, \dots, i'_n)$ be the optimal solution which has the fewest inversions. Letting j be the first index such that $i'_{j+1} < i'_j$, applying the exchange described above gives a solution $S'' = (i'_1, \dots, i'_{j-1}, i'_{j+1}, i'_j, i'_{j+2}, \dots, i'_n)$. Observe that S'' is optimal and has one fewer inversion than S' , which contradicts the assumption that S' is the optimal solution which has the fewest inversions. $\square$

2.3 Example 2: Cornucopia

Consider the following problem. You are at a feast where there are n courses. For each course you can choose one of the available dishes and you want to choose the tastiest dishes you can without choosing any dish twice. However, not all of the dishes are available right away. Instead, there are m dishes which will appear, each of which has an appearance time t_i when it becomes available and a value v_i representing how tasty it is. Given this, which dishes should you choose to maximize the total tastiness of the dishes you eat?

We can express this problem mathematically as follows. Given $\{(t_i, v_i) : i \in [m]\}$ (i.e., the appearance time and tastiness for each of the m dishes), choose a schedule $S = (i_1, \dots, i_n)$ for which dishes to eat such that

1. $i_1, \dots, i_n$ are distinct elements of $[m]$ (no dish is repeated twice).
2. For all $j \in [n]$, $t_{i_j} \leq j$ (Dish i_j is available at time j).

and $\sum_{j=1}^n v_{i_j}$ is maximized.

Note: We assume that $v_i \geq 0$ for all $i \in [m]$ and that $|\{i : t_i \leq j\}| \geq j$ for all $j \in [n]$ so that there is never a time when there are no dishes you would like to eat.

A greedy algorithm for this problem is to **choose the tastiest available dish**. More precisely, this algorithm works as follows:

1. Initialization: Choose i_1 so that $t_{i_1} \leq 1$ and $v_{i_1} = \max\{v_i : i \in [m], t_i \leq 1\}$ (i.e., choose the tastiest dish which is available at time 1).
2. Iterative step: If $j < n$ and $(i_1, \dots, i_j)$ is the schedule up to time j then choose i_{j+1} so that $i_{j+1} \in [m] \setminus \{i_1, \dots, i_j\}$, $t_{i_{j+1}} \leq j+1$, and $v_{i_{j+1}} = \max\{v_i : i \in [m] \setminus \{i_1, \dots, i_j\}, t_i \leq j+1\}$ (i.e., choose the tastiest available dish which you have not eaten already). If $j = n$ then we stop and output $S = (i_1, \dots, i_n)$.

We can prove that this algorithm is optimal as follows.

1. Finding an exchange: Let $S_{\text{greedy}} = (i_1, \dots, i_n)$ be the schedule given by the greedy algorithm and let $S' = (i'_1, \dots, i'_n)$ be a different schedule.

Let j be the first index such that $i'_j \neq i_j$. There are two cases to consider:

1. If there is an index $j' > j$ such that $i'_{j'} = i_j$ (i.e., dish i_j appears later on in S') then swap i'_j and $i'_{j'}$ to obtain the schedule $S'' = (i_1, \dots, i_j, i'_{j+1}, \dots, i'_{j'-1}, i'_j, i'_{j'+1}, \dots, i'_n)$. We know that this swap is valid as dish i_j is available at time j and since dish i'_j is available at time j , it will **still be available at time j'** .
2. If there is no index $j' > j$ such that $i'_{j'} = i_j$ (i.e., dish i_j does not appear later on in S') then we replace i'_j with i_j to obtain the schedule $S'' = (i_1, \dots, i_j, i'_{j+1}, \dots, i'_n)$.

2. Showing that the exchange can only improve S' : We make the following observations:

1. For the first case, **swapping two dishes does not affect the total tastiness of the dishes**.
2. For the second case, since $v_{i_j} = \max\{v_i : i \in [m] \setminus \{i_1, \dots, i_{j-1}\}, t_i \leq j\}$, $t_{i'_j} \leq j$, and $i'_j \in [m] \setminus \{i_1, \dots, i_{j-1}\}$, we must have that $v_{i_j} \geq v_{i'_j}$ so $\sum_{j'=1}^j v_{i_{j'}} + \sum_{j'=j+1}^n v_{i'_{j'}} \geq \sum_{j'=1}^{j-1} v_{i_{j'}} + \sum_{j'=j}^n v_{i'_{j'}}$. In other words, since S_{greedy} **chooses the tastiest available dish**, $v_{i_j} \geq v_{i'_j}$ so replacing i'_j with i_j can only improve S' .

3. Concluding that S_{greedy} is an optimal solution: Again, it would be acceptable to say something like “Since any other optimal solution can be made closer to the greedy solution without affecting its optimality, the greedy solution must be optimal.” That said, this is not quite rigorous.

There are several ways to rigorously prove that S_{greedy} is an optimal solution. One way to do this is to use induction to prove that for all $k \in [n] \cup \{0\}$, there is an optimal solution which matches the first k elements of the greedy solution. Applying this with $k = n$ proves that the greedy solution is optimal.

Lemma 2.8. *For all $k \in [n] \cup \{0\}$, there is an optimal solution $S' = (i'_1, \dots, i'_n)$ such that for all $j \in [k]$, $i'_j = i_j$ (i.e., the first j dishes in S' are the same as the first j dishes in S_{greedy}).*

Proof. The base case $k = 0$ is trivial. For the inductive step, assume that for some $k < n$, there is an optimal solution $S' = (i'_1, \dots, i'_n)$ such that for all $j \in [k]$, $i'_j = i_j$. If $i'_{k+1} = i_{k+1}$ then we are done. Otherwise, applying the exchange described above gives an optimal solution $S'' = (i''_1, \dots, i''_n)$ such that for all $j \in [k+1]$, $i''_j = i_j$. $\square$

Another way to prove that the greedy solution is optimal is to use a proof by contradiction using the lexicographic order.

Proof by contradiction using the lexicographic order. Relabel the dishes so that the greedy solution is $S_{greedy} = (1, \dots, n)$. Assume S_{greedy} is not an optimal solution and let $S' = (i'_1, \dots, i'_n)$ be the optimal solution which comes first in the lexicographic order. Letting j be the first index such that $i'_j \neq j$, applying the exchange described above gives a solution $S'' = (1, 2, \dots, j, i''_{j+1}, \dots, i''_n)$. Observe that S'' is optimal and comes before S' in the lexicographic order, which contradicts the assumption that S' is the optimal solution which comes first in the lexicographic order. $\square$

2.4 Common Exchanges Used in Exchange Arguments

As described at the beginning of this section, the key steps for an exchange argument are as follows:

1. Given a solution S' which is not the greedy solution, find an **exchange of two elements of S'** which makes S' **closer to the greedy solution**.
2. Show that this exchange **can only improve S'** .
3. Conclude that the greedy solution is optimal.

As illustrated by the above examples, two common exchanges which are sufficient are as follows. For simplicity, we assume that the elements are labeled so that the greedy solution is $S_{greedy} = (1, \dots, n)$ and that $(i_1, \dots, i_n)$ is another solution.

1. If $i_{j+1} < i_j$ then we can swap i_j and i_{j+1} .
2. Letting $j \in [n]$ be the smallest index such that $i_j \neq j$ (i.e., j is in the wrong place), we can swap i_j and j (or replace i_j with j if j does not appear).

The following lemmas justify the claim that either of these exchanges can be used in an exchange argument.

Lemma 2.9. *If we have that whenever $S = (i_1, \dots, i_n)$ is an optimal solution, $j \in [n - 1]$, and $i_{j+1} < i_j$, the solution $S' = (i_1, \dots, i_{j-1}, i_{j+1}, i_j, i_{j+2}, \dots, i_n)$ obtained by exchanging i_j and i_{j+1} is also optimal then the greedy solution $S_{\text{greedy}} = (1, 2, \dots, n)$ is optimal.*

There are several different ways to prove this lemma, two of which are as follows.

Proof by contradiction using the lexicographic order. Let $S = (i_1, \dots, i_n)$ be the optimal solution which comes first in the lexicographic order and assume that $S \neq S_{\text{greedy}}$. If so, there must be an index $j \in [n - 1]$ such that $i_{j+1} < i_j$. Letting S' be the optimal solution obtained by swapping i_j and i_{j+1} , S' comes before S in the lexicographic order which contradicts how we chose S . Thus, we must have that $S = S_{\text{greedy}}$. $\square$

Proof by induction using the number of inversions. An alternative proof of this lemma is as follows. We show that for all integers $k \in [0, \binom{n}{2}]$, there is an optimal solution with at most $\binom{n}{2} - k$ inversions (where an inversion is pair of indices $j, j' \in [n]$ such that $j < j'$ but $i_j > i_{j'}$).

The base case $k = 0$ is trivial. For the inductive step, if $k < \binom{n}{2}$, let S be a solution with $\binom{n}{2} - k$ inversions. Since S has at least one inversion, there must be an index $j \in [n - 1]$ such that $i_{j+1} < i_j$. Letting S' be the optimal solution obtained by swapping i_j and i_{j+1} , S' has one fewer inversion than S so S' is an optimal solution with $\binom{n}{2} - (k + 1)$ inversions.

Applying this result with $k = \binom{n}{2}$, there must be an optimal solution with 0 inversions, namely S_{greedy} . $\square$

Lemma 2.10. *If we have that whenever $S = (i_1, \dots, i_n)$ is an optimal solution and $j \in [n]$ is the first index such that $i_j \neq j$, the solution S' obtained by exchanging i_j and j (or replacing i_j with j if j does not appear in S) is also optimal then the greedy solution $S_{\text{greedy}} = (1, 2, \dots, n)$ is optimal.*

There are several different ways to prove this lemma, two of which are as follows.

Proof by contradiction. Let $S = (i_1, \dots, i_n)$ be the optimal solution which agrees with the maximum number k of initial terms of S_{greedy} . In other words, S maximizes the number k such that $i_j = j$ for all $j \in [k]$ and either $i_{k+1} \neq k + 1$ or $k = n$. Assume that $S \neq S_{\text{greedy}}$. If so, $k < n$ so letting S' be the optimal solution obtained by swapping i_{k+1} and $k + 1$ (or replacing i_{k+1} with $k + 1$ if $k + 1$ does not appear in S), S' matches the first $k + 1$ terms of S_{greedy} which contradicts how we chose S . Thus, we must have that $S = S_{\text{greedy}}$. $\square$

Proof by induction. We show that for all $k \in [n] \cup \{0\}$, there is an optimal solution which matches the first k terms of S_{greedy} (i.e., $i_j = j$ for all $j \in [k]$).

The base case $k = 0$ is trivial. For the inductive step, let S be a solution which matches the first k terms of S_{greedy} . If $k = n$ or $i_{k+1} = k + 1$ then we are done. Otherwise, letting S' be the optimal solution obtained by swapping i_{k+1} and $k + 1$ (or replacing i_{k+1} with $k + 1$ if $k + 1$ does not appear in S), S' is an optimal solution which matches the first $k + 1$ terms of S_{greedy} .

Applying this result with $k = n$, S_{greedy} must be an optimal solution. $\square$